Тестовое задание длянGolang-разработчика

Пакет TCP-сканирования портов

Необходимо разработать Go-пакет для проверки доступности TCP-портов на одном или нескольких хостах.

Пакет должен быть пригоден для встраивания в другие Go-приложения и не должен зависеть от конкретного CLI, HTTP-фреймворка, базы данных или брокера сообщений.

Ограничение задачи

Необходимо реализовать обычное TCP Connect-сканирование с использованием стандартных сетевых возможностей Go.

Не требуется реализовывать:

- SYN-сканирование;
- UDP-сканирование;
- raw sockets;
- определение операционной системы;
- обход межсетевых экранов;
- эксплуатацию уязвимостей;
- полноценный аналог Nmap.

Основная цель задания — спроектировать качественный Go-пакет, а не реализовать большое количество сетевых функций.

Общий сценарий использования

Пользователь пакета передаёт:

- один или несколько хостов;
- список или диапазон TCP-портов;
- количество одновременно выполняемых проверок;
- таймаут подключения;
- `context.Context`.

Пакет возвращает результат проверки каждого порта.

Возможный вариант использования:

```
scanner, err := portscan.New(
    portscan.WithConcurrency(100),
    portscan.WithConnectTimeout(500*time.Millisecond),
)
if err != nil {
    return err
}

results, err := scanner.Scan(
    ctx,
    []string{"192.168.1.10", "192.168.1.11"},
    portscan.Range(1, 1000),
)
if err != nil {
    return err
}

for result := range results {
    fmt.Printf(
        "%s:%d %s\n",
        result.Host,
        result.Port,
        result.State,
    )
}
```

Пример приведён только для понимания задачи. Конкретный публичный API кандидат проектирует самостоятельно.

Основные требования

1. Цели сканирования

Пакет должен поддерживать один или несколько хостов.

В качестве цели могут передаваться:

- IPv4-адрес;
- IPv6-адрес;
- DNS-имя.

Необходимо определить поведение для:

- пустого хоста;
- некорректного адреса;
- неразрешимого DNS-имени;
- повторяющихся целей;
- одного DNS-имени, которому соответствует несколько IP-адресов.

Поддержка CIDR необязательна и может быть реализована как дополнительная возможность.

2. Порты

Пакет должен поддерживать:

- список отдельных портов;
- диапазон портов;
- объединение нескольких диапазонов и отдельных портов.

Необходимо определить поведение для:

- порта `0`;
- порта больше `65535`;
- обратного диапазона, например `1000–1`;
- повторяющихся портов;
- пустого списка портов.

3. Состояния порта

Для каждого порта должен возвращаться структурированный результат.

Минимально необходимо различать:

- `open` — TCP-соединение установлено;
- `closed` — соединение явно отклонено;
- `timeout` — подключение не завершилось за установленное время;
- `unreachable` — узел или сеть недоступны;
- `canceled` — проверка отменена;
- `error` — произошла другая ошибка.

Не допускается определение состояния через сравнение текста ошибки.

4. Конкурентное выполнение

Проверки портов должны выполняться конкурентно.

Максимальное количество одновременно выполняемых подключений должно настраиваться.

Необходимо корректно обработать:

- большое количество целей и портов;
- медленные подключения;
- отмену через `context.Context`;
- остановку отправки новых заданий после отмены;
- завершение уже запущенных операций;
- закрытие каналов с результатами;
- ситуацию, когда пользователь медленно читает результаты;
- повторное использование одного экземпляра сканера.

5. Получение результатов

Результат проверки должен содержать как минимум:

```
type Result struct {
    Host     string
    IP       net.IP
    Port     uint16
    State    State
    Duration time.Duration
    Err      error
}
```

Эта структура приведена только как пример.

Архитектурные требования

Пакет должен:

- иметь понятный публичный API;
- не экспортировать лишние сущности;
- не использовать глобальное изменяемое состояние;
- корректно работать при конкурентном использовании;
- поддерживать отмену через `context.Context`;
- ограничивать количество одновременно выполняемых операций;
- корректно закрывать созданные TCP-соединения;
- возвращать структурированные результаты и ошибки;
- проходить команду:

go test -race ./...

README

В README необходимо описать:

1. Как использовать пакет.
2. Как устроен публичный API.

Ограничения

- Go версии 1.25.5.
- Разрешено использовать стандартную библиотеку Go.
- Сторонние зависимости допускаются, но их необходимость требуется обосновать.
- Код должен быть размещён в Git-репозитории.
- Пакет должен собираться и тестироваться командой:

go test -race ./...